BUTTON AND SWITCH LOGIC

Curiosity Kickoff

- What happens when you turn ON a switch?
- Does it turn OFF automatically?
- Why does a light stay ON after switching?

Concept Connection

This topic introduces students to Practice More with Pins through the session Button and Switch Logic. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

1.Potentiometer

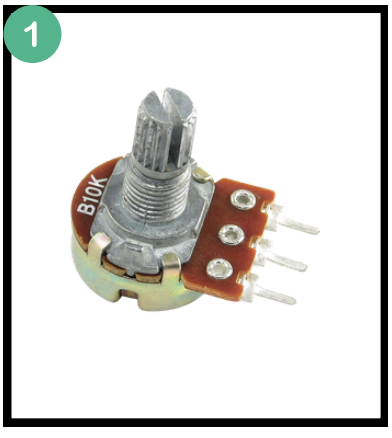
A Potentiometer is a variable resistor that allows you to adjust the amount of electrical resistance in a circuit. By rotating its knob, you can control values such as brightness, volume, or motor speed in electronic projects.

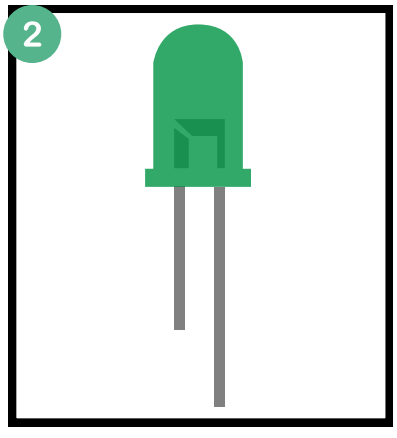
2. Dip LED

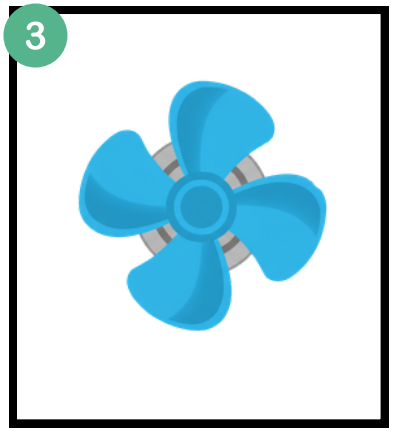
A DIP LED (Dual In-line Package LED) is a light-emitting diode that produces light when an electric current passes through it. It is commonly used as a status indicator, signal light, or decorative lighting in electronic circuits and projects.

3. DC Motor

A DC Motor is an electrical device that converts direct current (DC) into mechanical motion. When powered, it rotates and is commonly used in fans, toys, robots, and small machines.







Beginner: Toggle Switch Basics

Class Explanation

In this class, students learn the basic idea of Toggle Switch Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × switch pin
- 1 × Red 1W SMD LED on breakout board
- RMC wires
- USB cable

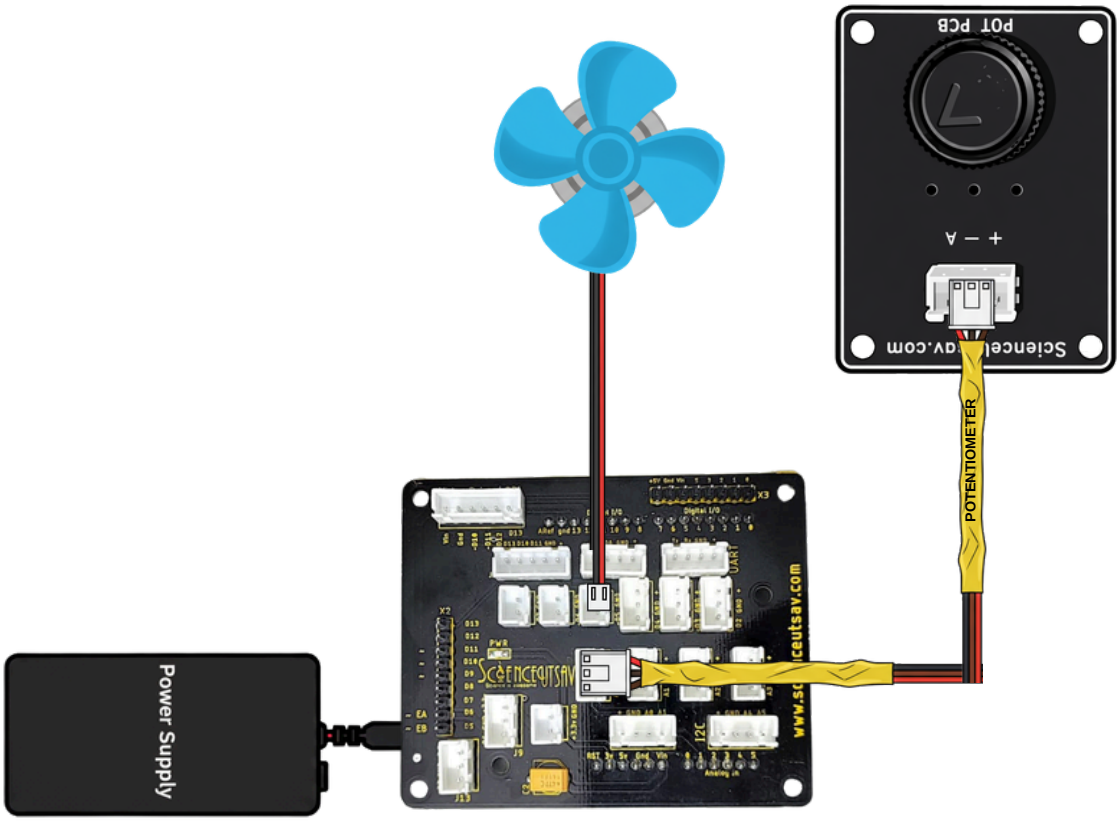
How to Connect

Use RMC wires to connect :

- 1 × switch pin→ Arduino pin A0 and GND
- 1 × Red 1W SMD LED → Arduino pin A0 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int potPin = A0;      // Potentiometer connected to A0
const int motorPin = 5;     // PWM pin connected to motor driver

void setup() {
  pinMode(motorPin, OUTPUT);
}

void loop() {
  // Read the potentiometer value (0 to 1023)
  int potValue = analogRead(potPin);

  // Convert it to PWM value (0 to 255)
  int motorSpeed = map(potValue, 0, 1023, 0, 255);

  // Set motor speed
  analogWrite(motorPin, motorSpeed);

  delay(10);
}
```

Code Explanation

- In this Arduino program, a potentiometer is connected to analog pin A0 (potPin) and a DC motor is connected to PWM pin 5 (motorPin) through a motor driver or transistor.
- The setup() function runs once when the Arduino starts. It sets the motor pin as an OUTPUT so the Arduino can control the motor's speed.
- The loop() function runs continuously. It reads the position of the potentiometer using analogRead(), which gives a value between 0 and 1023 depending on how much the knob is rotated.
- The map() function converts this value to a range of 0 to 255, which is required for PWM (Pulse Width Modulation). The analogWrite() function then sends this PWM signal to the motor, controlling its speed.
- So overall, rotating the potentiometer changes the speed of the DC motor. Turning the knob one way slows the motor down, while turning it the other way increases the motor's speed.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Potentiometer Basics but change the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int potPin = A0;
const int ledPin = 5;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (analogRead(potPin) > 512)
    digitalWrite(ledPin, HIGH);
  else
    digitalWrite(ledPin, LOW);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:

The basic idea of Poentiometer, how the code connects with the circuit behavior, how to upload and test a simple Arduino sketch

Advanced: Logic Gates with Switches

Class Explanation

In this class, students learn the basic idea of Logic Gates with Switches. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 2 × Switches
- 2 × Red 1W SMD LED on breakout boardRMC wires
- USB cable

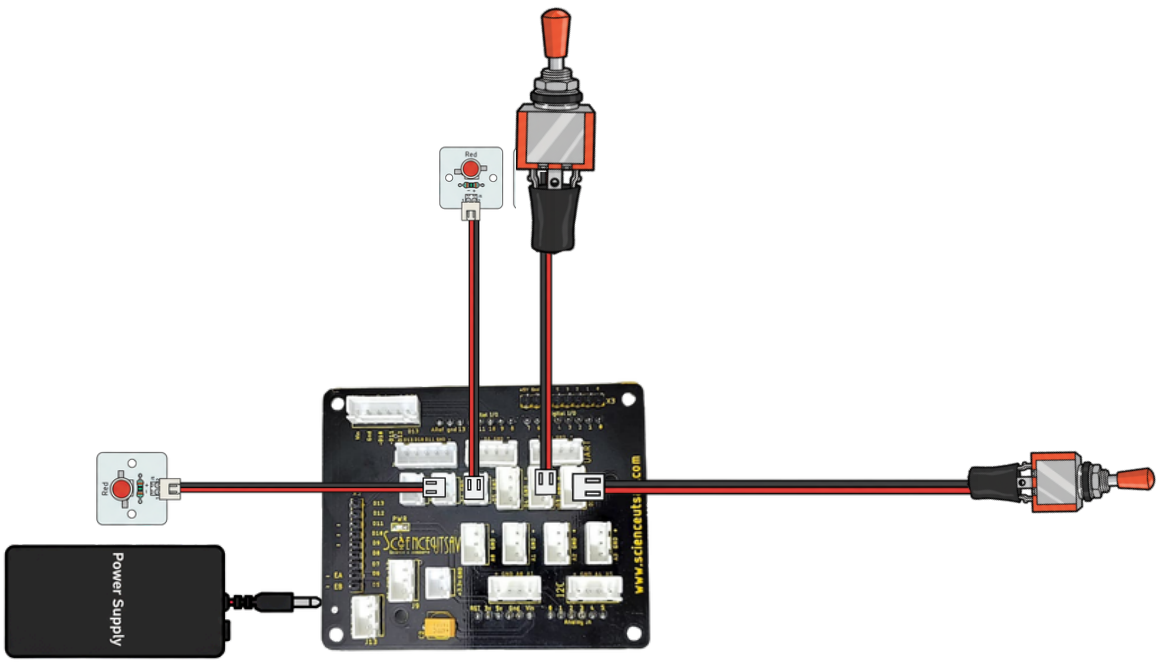
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- 1 × Switch → Arduino pin D2 and GND
- 1 × Switch → Arduino pin D3 and GND
- 1 × Red 1W SMD LED → Arduino pin D5 and GND
- 1 × Red 1W SMD LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int switch1 = 2;
const int switch2 = 3;
const int andLed = 5;
const int orLed = 6;

void setup() \
 pinMode(switch1, INPUT_PULLUP);
 pinMode(switch2, INPUT_PULLUP);
 pinMode(andLed, OUTPUT);
 pinMode(orLed, OUTPUT);
\

void loop() \
 bool a = digitalRead(switch1) == LOW;
 bool b = digitalRead(switch2) == LOW;
 digitalWrite(andLed, a && b);
 digitalWrite(orLed, a || b);
\
```

Code Explanation

- In this Arduino program, two switches are connected to pins 2 (switch1) and 3 (switch2), and two LEDs are connected to pins 5 (andLed) and 6 (orLed).
- The setup() function runs once when the Arduino starts. It sets both switches as INPUT_PULLUP and both LEDs as OUTPUT.
- The loop() function runs continuously. It reads both switch states using digitalRead(). Since INPUT_PULLUP is used, a pressed switch gives a LOW signal, so boolean values a and b become true when switches are pressed.
- The LED on pin 5 works as an AND gate. It turns ON only when both switches are pressed.
- The LED on pin 6 works as an OR gate. It turns ON when at least one switch is pressed.

So overall, the program demonstrates AND and OR logic using switches and LEDs.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Laser LDR Beam Break Detection but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int switch1 = 2;
const int switch2 = 3;
const int andLed = 5;
const int orLed = 6;
const int xorLed = 7;

void setup() \
 pinMode(switch1, INPUT_PULLUP);
 pinMode(switch2, INPUT_PULLUP);
 pinMode(andLed, OUTPUT);
 pinMode(orLed, OUTPUT);
 pinMode(xorLed, OUTPUT);
\

void loop() \
 bool a = digitalRead(switch1) == LOW;
 bool b = digitalRead(switch2) == LOW;

 digitalWrite(andLed, a && b);
 digitalWrite(orLed, a || b);
 digitalWrite(xorLed, a != b);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Logic Gates with Switches how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Two Way Switch Wiring and Logic

Class Explanation

In this class, students learn the basic idea of Two Way Switch Wiring and Logic. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 2 × Switches
- 1 × Red 1W SMD LED on breakout board
- RMC wires
- USB cable

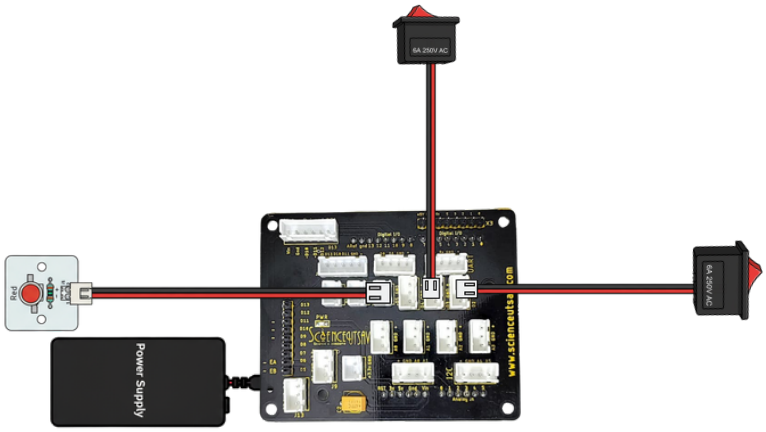
How to Connect

Use RMC wires to connect:

- 1 × Switches→ Arduino pin D2 and GND
- 1 × Switches→ Arduino pin D3 and GND
- 1 × Red 1W SMD LED → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int switch1 = 2;
const int switch2 = 3;
const int ledPin = 5;

void setup() \
 pinMode(switch1, INPUT_PULLUP);
 pinMode(switch2, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 bool a = digitalRead(switch1) == LOW;
 bool b = digitalRead(switch2) == LOW;
 digitalWrite(ledPin, a != b);
\
```

Code Explanation

- In this Arduino program, two switches are connected to pins 2 (switch1) and 3 (switch2), and an LED is connected to pin 5 (ledPin).
- The setup() function runs once when the Arduino starts. It sets both switches as INPUT_PULLUP and the LED as OUTPUT.
- The loop() function runs continuously. It reads the states of both switches using digitalRead(). Since INPUT_PULLUP is used, a pressed switch gives a LOW signal, so the variables a and b become true when the switches are pressed.
- The condition a != b represents an XOR (Exclusive OR) operation. The LED turns ON only when exactly one switch is pressed, and turns OFF when both are pressed or both are not pressed.

So overall, the program demonstrates XOR logic using two switches and one LED.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Two Way Switch Wiring and Logic but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int switch1 = 2;
const int switch2 = 3;
const int ledPin = 5;

void setup() \
 pinMode(switch1, INPUT_PULLUP);
 pinMode(switch2, INPUT_PULLUP);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 bool firstState = digitalRead(switch1) == LOW;
 bool secondState = digitalRead(switch2) == LOW;

 if (firstState != secondState) \
   digitalWrite(ledPin, HIGH);
 \ else \
   digitalWrite(ledPin, LOW);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Two Way Switch Wiring and Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes

- the basic idea of Toggle Switch Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Limit Switch Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Logic Gates with Switches how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Two Way Switch Wiring and Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself codecode
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output